

TEST ASSESSMENT

Q1, Write a function to find the longest common prefix string amongst an array of strings.
If there is no common prefix, return an empty string "".

Example 1:

input: strs = ["flower", "flow", "flight"]

output: "fl"

Example 2:

input: strs = ["dog", "racecar", "car"]

output: ""

Explanation: there is no common prefix among the input strings.

Java Code:

```
import java.util.*;
```

```
class Main{
```

```
    public static String longestCommonPrefix(String[] strs){
```

```
        if(strs == null || strs.length == 0){
```

```
            return "";
```

```
        }
```

```
        String prefix = strs[0];
```

```
        for(int i = 1; i < strs.length; i++){
```

```
            while (!strs[i].startsWith(prefix)){
```

```
                prefix = prefix.substring(0, prefix.length() - 1);
```

```
                if(prefix.isEmpty()){
```

```
                    return "";
```

```
            }
```

```
        }
```

```
    }
```

```
    return prefix;
```

```
}
```

```

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    String[] strs = new String[n];
    System.out.println("Enter the strings:");
    for(int i=0; i<n; i++){
        strs[i] = sc.next();
    }
    String result = longestCommonPrefix(strs);
    System.out.println("Longest common Prefix: "+result);
    sc.close();
}
}

```

Expected input:-

3

Enter the strings:

flower

flow

fright

output:

fl

Q2) You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0. Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to an index `(i+j)` where:

- $0 \leq j \leq \text{nums}[i]$ and
- $i+j < n$

Return the minimum number of jumps to reach index `n-1`. The test cases are generated such that you can reach index `n-1`.

Example 1:

input: `nums = [2, 3, 1, 1, 4]`

output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

input: `nums = [2, 3, 0, 1, 4]`

output: 2

Java code:

```
import java.util.*;
class Main {
    public static int minJumps(int[] nums) {
        if (nums == null || nums.length <= 1) {
            return 0;
        }
        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
        }
    }
}
```

```

        if (i == currentEnd) {
            jumps++;
            currentEnd = -farthest;
            if (currentEnd >= nums.length - 1) {
                break;
            }
        }
    }
}
return jumps;
}

```

```

}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int[] nums = new int[n];
    for (int i = 0; i < n; i++) {
        nums[i] = sc.nextInt();
    }
    int result = minJump(nums);
    System.out.println(result);
    sc.close();
}
}

```

Input :-

5
2 3 1 1 4

Output :-

2